

Compte rendu n°4
Tp Tests de Logiciels
ChatGPT pour faciliter l'automatisation des tests logiciel

Nom:Rayen Bouyahia
Classe:L2CS01

1. Introduction

Ce TP explore l'utilisation de ChatGPT (basé sur le modèle GPT-3.5) pour faciliter l'automatisation des tests logiciel. L'idée de départ est simple : si ChatGPT peut écrire du code dans différents langages, peut-il nous aider à construire un pipeline de test complet, du script unitaire au workflow CI/CD ?

Le TP suit une progression logique, en partant d'un script Selenium basique et en enrichissant progressivement le pipeline : scénarios Cucumber, Playwright, requêtes MySQL, résilience, GitHub Actions et script Bash de déploiement.

2. Premier script Selenium avec ChatGPT

La première expérimentation consiste à demander à ChatGPT de générer un script Selenium en Java capable d'ouvrir Google.com et d'afficher son titre, tout en étant compatible avec la grille cloud LambdaTest.

Prompt utilisé :

"Écrivez un code Selenium et Java pour ouvrir google.com et imprimer son titre. Le code doit s'exécuter sur LambdaTest"

Résultat — ChatGPT a généré un code complet incluant automatiquement les éléments LambdaTest (hub URL, DesiredCapabilities, clés d'accès) sans qu'on ait besoin de le préciser :

```

1 import org.openqa.selenium.WebDriver;
2 import org.openqa.selenium.chrome.ChromeDriver;
3 import org.openqa.selenium.remote.RemoteWebDriver;
4 import org.openqa.selenium.remote.DesiredCapabilities;
5
6 public class Test {
7     public static void main(String[] args) {
8         // Set your LambdaTest username and access key
9         String username = "YOUR_USERNAME";
10        String accessKey = "YOUR_ACCESS_KEY";
11
12        // Set the desired browser and operating system
13        DesiredCapabilities caps = new DesiredCapabilities();
14        caps.setCapability("browserName", "chrome");
15        caps.setCapability("version", "80.0");
16        caps.setCapability("platform", "WIN10");
17
18        // Create a webdriver instance using the LambdaTest grid
19        WebDriver driver = new RemoteWebDriver(
20            new java.net.URL("https://" + username + ":" + accessKey + "@hub.lambdatest.com/wd/hub"),
21            caps
22        );
23
24        // Open Google.com
25        driver.get("https://www.google.com");
26
27        // Get the title of the page
28        String title = driver.getTitle();
29
30        // Print the title
31        System.out.println(title);
32
33        // Close the webdriver instance
34        driver.quit();
35    }
36 }

```

Figure 1 — Code Selenium/Java généré par ChatGPT pour LambdaTest

Ce premier résultat est convaincant : le code est structuré, commenté et prêt à être adapté. On voit que ChatGPT a bien compris le contexte LambdaTest et a inclus les dépendances nécessaires.

3. Scénarios Cucumber — cas positifs et négatifs

3.1 Scénario complexe : plateforme à 4 microservices

On passe à un cas plus réaliste : une plateforme Selenium composée de 4 microservices (hub_service, test_management_service, linux_container_service, log_management_service). ChatGPT a produit une définition Cucumber structurée couvrant le flux principal.

3.2 Ajout des cas négatifs

Sans les demander explicitement dans la première requête, les cas négatifs étaient absents. En relançant un prompt ciblé ("Générer des tests négatifs lorsque test_service et linux_container_service échouent"), ChatGPT a complété la définition avec les scénarios d'erreur :

Figure 2 — Scénarios négatifs générés pour test_management_service et linux_container_service

4. Implémentation Playwright

Une fois les cas Cucumber définis, on demande à ChatGPT de générer l'implémentation Playwright correspondante. Le résultat couvre les scénarios positifs et négatifs pour les 4 microservices, avec la structure beforeEach/afterEach pour la gestion du navigateur :

```
1  const { playwright } = require('playwright');
2
3  // define test variables
4  const TEST_URL = 'https://www.example.com';
5  const TEST_LOGS_API = 'https://www.logs.com/api';
6
7  describe('Selenium testing platform', () => {
8    let browser;
9    let context;
10   let page;
11
12   beforeEach(async () => {
13     // launch browser and create new page
14     browser = await playwright.chromium.launch();
15     context = await browser.newContext();
16     page = await context.newPage();
17   });
18
19   afterEach(async () => {
20     // close browser
21     await browser.close();
22   });
23
24   it('should successfully run a Selenium test', async () => {
25     // send test to hub_service
26     await page.goto(TEST_URL);
27     // create test entity in database using test_management_service
28     // connect to linux_container_service to get docker container
29     // proxy HTTP requests to docker container
30     // collect test execution logs using log_management_service
31     const logs = await page.evaluate(() => fetch(TEST_LOGS_API).then(res => res.json()));
32
33     // assert that test ran successfully and logs are retrieved
34     expect(logs).toBeDefined();
35   });
36
37   it('should not run test if test_management_service fails', async () => {
38     // send test to hub_service
39     await page.goto(TEST_URL);
40     // simulate failure of test_management_service
41     const errorMessage = await page.evaluate(() => {
42       try {
43         // create test entity in database using test_management_service
44         throw new Error('test_management_service failure');
45       } catch (error) {
46         return error.message;
47       }
48     });
49
50     // assert that test did not run and error message is displayed
51     expect(errorMessage).toEqual('test_management_service failure');
52   });
53
54   it('should not run test if linux_container_service fails', async () => {
55     // send test to hub_service
56     await page.goto(TEST_URL);
57     // create test entity in database using test_management_service
```

Figure 3 — Implémentation Playwright générée pour les scénarios Cucumber (positifs et négatifs)

L'adaptation pour LambdaTest a ensuite été demandée en une seule question supplémentaire : ChatGPT a modifié la variable TEST_URL et ajouté la configuration des credentials via setExtraHTTPHeaders.

5. Vérification MySQL et résilience

5.1 Requête MySQL dans les tests

Pour vérifier que le `test_management_service` crée bien l'entité en base, on demande à ChatGPT d'ajouter une connexion MySQL et une requête SELECT dans le cas de test. Le package `mysql` npm est utilisé, la connexion est ouverte et fermée proprement dans le test.

5.2 Ajout de la résilience (retry)

Problème réaliste : si `test_management_service` est temporairement indisponible, le test doit réessayer automatiquement. ChatGPT a intégré une boucle de retry avec `maxRetries = 3` et un délai de 1 seconde entre chaque tentative :

resilience. ChatGPT, seriez-vous si gentil ?

- Q : Ajoutez de la résilience dans les cas de test ci-dessus où `test_management_service` devient temporairement indisponible en ajoutant des tentatives.
- ChatGPT : pour ajouter de la résilience aux cas de test ci-dessus où le `test_management_service` peut devenir temporairement indisponible, vous pouvez ajouter des tentatives en utilisant une boucle et un délai entre chaque nouvelle tentative.
- Voici un exemple de la façon dont cela peut être fait :

```
1 const mysql = require('mysql');
2
3 describe('Selenium testing platform', () => {
4   // ... existing test cases ...
5
6   it('should verify test entity in database with retries', async () => {
7     let result;
8     let retries = 0;
9     const maxRetries = 3;
10    const retryDelay = 1000; // 1 second
11
12    while (!result && retries < maxRetries) {
13      // send test to hub_service
14      await page.goto(HUB_SERVICE_URL);
15      // create test entity in database using test_management_service
16      await page.goto(TEST_MANAGEMENT_SERVICE_URL);
17      // query database to verify test entity was created
18      result = await page.evaluate(() => {
19        // create MySQL connection
20        const connection = mysql.createConnection({
21          host: 'localhost',
22          user: 'your_username',
23          password: 'your_password',
24          database: 'your_database',
25        });
26
27        // execute query to retrieve test entity
```

Figure 4 — Logique de retry avec connexion MySQL et gestion de l'indisponibilité du service

5.3 Sécurisation via variables d'environnement

Les credentials MySQL (host, user, password) ont ensuite été déplacés dans des variables d'environnement (process.env.MYSQL_HOST, etc.) pour éviter de les exposer en clair dans le code — bonne pratique de sécurité que ChatGPT applique sur demande.

6. Pipeline CI/CD avec GitHub Actions

L'étape finale consiste à automatiser l'ensemble du pipeline. ChatGPT génère un fichier YAML GitHub Actions qui déclenche les tests tous les jours à 9h00, pointe vers LambdaTest, et récupère les résultats via l'API LambdaTest :

```
5 - cron: '0 9 * * *'
6
7 jobs:
8   test:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v2
12      - name: Set up Node.js
13        uses: actions/setup-node@v2
14        with:
15          node-version: 14.x
16      - name: Install dependencies
17        run: npm install
18      - name: Run tests
19        env:
20          TEST_URL: https://hub.lambdatest.com/wd/hub
21          USERNAME: ${ secrets.LAMBDATEST_USERNAME }
22          ACCESS_KEY: ${ secrets.LAMBDATEST_ACCESS_KEY }
23        run: npm test
```

- Cette action est déclenchée par un événement programmé tous les jours à 9h00, comme spécifié sur le bloc. Il effectue les étapes suivantes :
- Consultez le référentiel à l'aide de l'action actions/checkout.

Figure 5 — Workflow GitHub Actions généré : exécution quotidienne à 9h00 sur LambdaTest

Les secrets LambdaTest (USERNAME, ACCESS_KEY) sont injectés via les GitHub Secrets — sécurité intégrée par défaut dans la réponse de ChatGPT, sans qu'on ait besoin de le demander explicitement cette fois.

ChatGPT a également généré un fichier package.json avec les scripts test, fetch-results et run-tests, ainsi qu'un script Bash complet pour cloner le repo, installer les dépendances et configurer tout l'environnement en local sur Mac.

7. Bilan : avantages et limites

7.1 Points forts

- **Gain de temps significatif sur la génération de code boilerplate (Selenium, Cucumber, Playwright, YAML).**
- **Bonne gestion de contextes complexes : ChatGPT a retenu les microservices d'une question à l'autre.**
- **Facilite la montée en compétence : utile pour un développeur Selenium qui ne maîtrise pas GitHub Actions.**
- **Code sécurisé par défaut dans plusieurs cas (GitHub Secrets, variables d'env sur demande).**

7.2 Limites identifiées

Limite : Code incomplet : ChatGPT génère parfois des scripts tronqués. Il a fallu relancer avec "Remplissez le fichier Bash ci-dessus" pour obtenir la version complète.

Limite : Dépendance aux statistiques : GPT-3.5 prédit les mots selon leur contexte sans réellement "comprendre" le code. Des méthodes dépréciées peuvent apparaître si le modèle n'a pas été mis à jour.

Limite : Hypothèses implicites : ChatGPT fait des suppositions sur le contexte (URLs, noms de tables) qui peuvent être incorrectes et nécessitent une validation humaine.

8. Conclusion

Ce TP démontre que ChatGPT est un outil genuinely utile pour l'automatisation des tests, notamment pour accélérer la génération de code répétitif et aider à franchir des étapes techniques inconnues. En partant d'un simple script Selenium, on a construit un pipeline complet : Cucumber, Playwright, MySQL, retry, GitHub Actions et script Bash.

Cependant, ChatGPT ne remplace pas l'ingénieur QA. Le code produit nécessite toujours une validation, une complétion et parfois une correction. C'est avant tout un assistant qui fait gagner du temps sur les tâches de scaffolding, pas un expert autonome.